

Feed Forward Backpropagation Neural Network

AIM : Implement the Feed Forward Backpropagation algorithm to train a neural network.

B) Use a given dataset to train the neural network for a specific task.

C) Evaluate the performance of the trained network on test data.

Code :

```
ai p4 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p4 T117.py (3.12.6)
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Adarsh Shukla T117")

df = pd.read_csv(r"C:\Users\Adarsh\Downloads\diabetes997.csv")

print("\nDataset:")
print(df.head())

if "Outcome" in df.columns:
    target = "Outcome"
elif "outcome" in df.columns:
    target = "outcome"
elif "Diabetes" in df.columns:
    target = "Diabetes"
else:
    target = df.columns[-1]

X = df.drop(columns=[target])
y = df[target]

X = X.select_dtypes(include=[np.number])
X = X.fillna(X.mean())

if y.dtype == "object":
    y = pd.factorize(y)[0]
```

T117 adarsh shukla

Roll no : t117

Sub : Artificial Intelligence

Feed Forward Backpropagation Neural Network

```
ai p4 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p4 T117.py (3,12.8)
File Edit Format Run Options Window Help

X = X.select_dtypes(include=[np.number])
X = X.fillna(X.mean())

if y.dtype == "object":
    y = pd.factorize(y)[0]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = np.array(X_train)
X_test = np.array(X_test)

y_train = np.array(y_train).reshape(-1, 1)
y_test = np.array(y_test).reshape(-1, 1)

np.random.seed(42)

input_neurons = X_train.shape[1]
hidden_neurons = 8
output_neurons = 1
```

T117 adarsh shukla

Roll no : t117

Sub : Artificial Intelligence

Feed Forward Backpropagation Neural Network

```
ai p4 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p4 T117.py (3.12.6)
File Edit Format Run Options Window Help
output_neurons = 1

W1 = np.random.randn(input_neurons, hidden_neurons) * 0.1
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons) * 0.1
b2 = np.zeros((1, output_neurons))

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_derivative(x):
    return x * (1 - x)

learning_rate = 0.1
epochs = 5000

loss_history = []

for epoch in range(epochs):

    hidden_input = np.dot(X_train, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    error = y_train - final_output

    loss = np.mean(error ** 2)
    loss_history.append(loss)

    output_delta = error * sigmoid_derivative(final_output)
    hidden_error = np.dot(output_delta, W2.T)
    hidden_delta = hidden_error * sigmoid_derivative(hidden_output)

    W2 += learning_rate * np.dot(hidden_output.T, output_delta) / len(X_train)
    b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)

    W1 += learning_rate * np.dot(X_train.T, hidden_delta) / len(X_train)
    b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    if epoch % 1000 == 0:
        print("Epoch:", epoch, "Loss:", round(loss, 4))

hidden_test = sigmoid(np.dot(X_test, W1) + b1)
test_output = sigmoid(np.dot(hidden_test, W2) + b2)
predictions = (test_output >= 0.5).astype(int)

print("\nActual:")
print(y_test.flatten())

print("\nPredicted:")
print(predictions.flatten())

accuracy = accuracy_score(y_test, predictions)

print("\nAccuracy:", round(accuracy * 100, 2), "%")
```

T117 adarsh shukla

Roll no : t117

Sub : Artificial Intelligence

Feed Forward Backpropagation Neural Network

```
ai p4 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p4 T117.py (3.12.6)
File Edit Format Run Options Window Help

print("Epoch:", epoch, "Loss:", round(loss, 4))

hidden_test = sigmoid(np.dot(X_test, W1) + b1)
test_output = sigmoid(np.dot(hidden_test, W2) + b2)
predictions = (test_output >= 0.5).astype(int)

print("\nActual:")
print(y_test.flatten())

print("\nPredicted:")
print(predictions.flatten())

accuracy = accuracy_score(y_test, predictions)

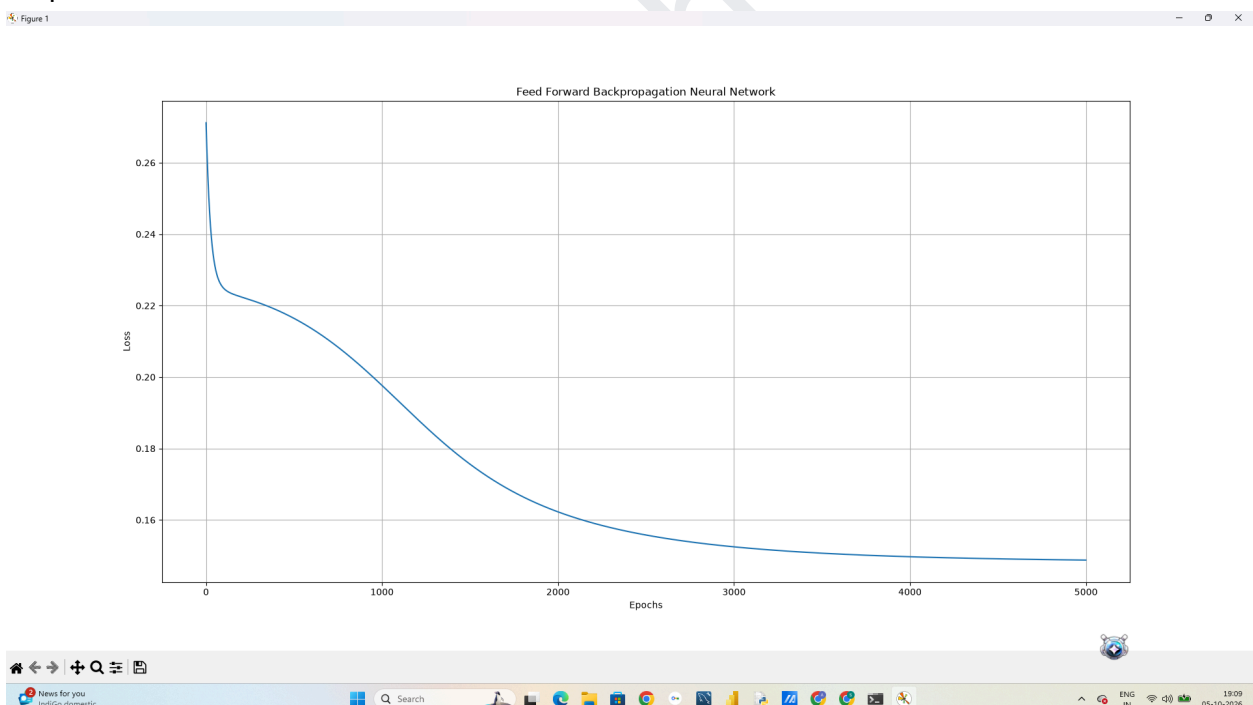
print("\nAccuracy:", round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

print("\nClassification Report:")
print(classification_report(y_test, predictions))

plt.figure(figsize=(8, 5))
plt.plot(loss_history)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Feed Forward Backpropagation Neural Network")
plt.grid()
plt.show()
```

Output :



T117 adarsh shukla

Roll no : t117

Sub : Artificial Intelligence